

SparkClean

Product Requirements Document

Backend: Prisma ORM + PostgreSQL + Node.js (Express/Fastify)

Version	1.0.0	Status	Draft → Review
Date	April 2026	Owner	VRC Pvt Ltd
Platform	Web (React 19 + TypeScript)	Region	Vizag, India
Contact	9392420643	Email	Welcome@vrcpvtltd.com

This document defines the complete product requirements, backend architecture, database schema, API design, and implementation guide for SparkClean — Visakhapatnam's premier home cleaning booking platform. Built on Prisma ORM + PostgreSQL with a Node.js REST API backend.

Table of Contents

1.	Executive Summary & Business Overview
2.	Tech Stack Architecture
3.	Prisma + PostgreSQL Database Schema
4.	REST API Design & Endpoints
5.	Authentication & Authorization
6.	Payment Integration (Cashfree)
7.	Services & Pricing Catalogue
8.	Booking Flow & Business Logic
9.	Admin Dashboard Backend
10.	Notifications (WhatsApp + Email)
11.	Security & Middleware
12.	Deployment Architecture
13.	Environment Variables
14.	Antigravity Prompt

1. Executive Summary & Business Overview

Product Vision

SparkClean is Visakhapatnam's first premium, tech-enabled home cleaning platform. Inspired by Urban Company and Pronto, SparkClean targets premium areas (MVP Colony, Madhurawada) while serving all of Vizag. The platform enables instant service booking, transparent pricing, Cashfree payments, and real-time admin management.

Business Goals

Goal	Target	Timeline
Launch MVP in Vizag	2 premium areas	Month 1
500+ bookings	MVP Colony + Madhurawada	Month 3
4.5+ average rating	All completed bookings	Month 2
Zero payment failures	Cashfree integration	Launch
Sub-2hr booking confirmation	All booking types	Launch

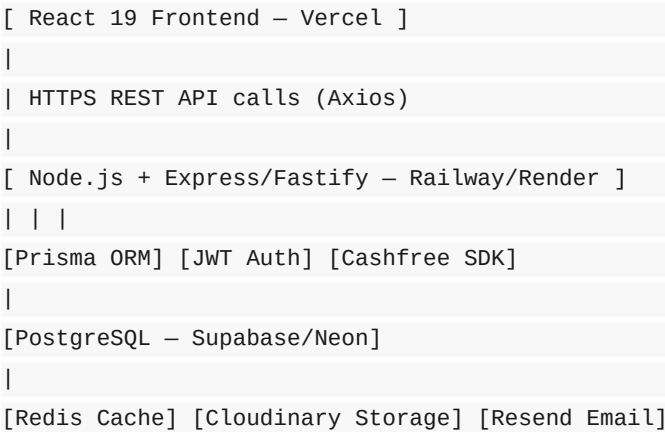
User Roles

Role	Description	Access Level
Customer	Homeowners booking cleaning services	Own bookings, profile, payments
Admin	SparkClean operations team	Full platform access + analytics
Cleaner	Assigned cleaning staff	Assigned jobs, status updates

2. Tech Stack Architecture

Layer	Technology	Version	Purpose
Frontend	React + TypeScript	19 / 5+	UI, routing, state
Styling	Tailwind CSS	3.x	Utility-first CSS
Animation	Framer Motion + GSAP	Latest	Page transitions, 3D hero
API Client	Axios + React Query	Latest	HTTP calls + caching
Backend Runtime	Node.js	20 LTS	Server runtime
Framework	Express.js / Fastify	4.x / 5.x	REST API server
ORM	Prisma ORM	5.x	Type-safe DB queries
Database	PostgreSQL	16	Primary data store
Auth	JWT + bcrypt	Latest	Stateless auth tokens
File Storage	Cloudinary / S3	Latest	Invoice PDFs, images
Email	Resend / Nodemailer	Latest	Booking confirmations
Payments	Cashfree PG	2023-08-01	UPI, cards, wallets
Caching	Redis (optional)	7.x	Session + rate limiting
Deployment	Railway / Render	Latest	Backend hosting
Frontend Deploy	Vercel	Latest	React app hosting
DB Hosting	Supabase / Neon	Latest	Managed PostgreSQL

Architecture Diagram



prisma/schema.prisma

[illegible]

[illegible]

[illegible]

```
createdAt DateTime @default(now())
```

```
}
```


4. REST API Design & Endpoints

Auth Routes — /api/auth

Method	Endpoint	Description	Auth
POST	/register	Create new customer account	Public
POST	/login	Login, returns JWT token	Public
POST	/logout	Invalidate token	JWT
POST	/forgot-password	Send reset email	Public
POST	/reset-password	Set new password	Token
GET	/me	Get current user profile	JWT

Booking Routes — /api/bookings

Method	Endpoint	Description	Auth
POST	/	Create new booking	JWT
GET	/	Get my bookings (paginated)	JWT
GET	/:id	Get single booking details	JWT
PATCH	/:id/cancel	Cancel a booking	JWT
POST	/:id/review	Submit rating + review	JWT
GET	/admin/all	Get all bookings (admin)	Admin
PATCH	/admin/:id/status	Update booking status	Admin
PATCH	/admin/:id/assign	Assign cleaner	Admin

Services Routes — /api/services

Method	Endpoint	Description	Auth
GET	/	Get all active services	Public
GET	/combos	Get combo packages only	Public
POST	/	Create service (admin)	Admin
PATCH	/:id	Update service (admin)	Admin
DELETE	/:id	Deactivate service (admin)	Admin

Payment Routes — /api/payments

Method	Endpoint	Description	Auth
POST	/create-order	Create Cashfree order	JWT
POST	/webhook	Cashfree payment webhook	Cashfree
GET	/:bookingId/status	Check payment status	JWT
POST	/refund	Process refund (admin)	Admin

Admin Routes — /api/admin

Method	Endpoint	Description	Auth
GET	/stats	Dashboard KPI metrics	Admin
GET	/revenue	Revenue by day/week/month	Admin
GET	/top-areas	Bookings by area heatmap	Admin
GET	/top-services	Most booked services	Admin
GET	/customers	All customer list	Admin
PATCH	/customers/:id/role	Change user role	Admin

Promo Routes — /api/promo

Method	Endpoint	Description	Auth
POST	/validate	Validate promo code	JWT
POST	/	Create promo code (admin)	Admin
GET	/	List all promo codes (admin)	Admin
DELETE	/:id	Delete promo code (admin)	Admin

5. Authentication & Authorization

JWT Token Strategy

SparkClean uses stateless JWT authentication. Tokens are signed with a secret key and carry the user's ID, email, and role. Access tokens expire in 7 days. A refresh token strategy can be added in v2.

```
POST /api/auth/login
```

```
Body: { email, password }
```

1. Find user by email in PostgreSQL via Prisma
2. Compare password with `bcrypt.compare()`
3. If valid: sign JWT with { `userId`, `email`, `role` }
4. Return: { `token`, `user`: { `id`, `name`, `email`, `role` } }

Protected routes use middleware:

```
const authenticate = (req, res, next) => {  
  const token = req.headers.authorization?.split(" ")[1]  
  if (!token) return res.status(401).json({ error: 'Unauthorized' })  
  const decoded = jwt.verify(token, process.env.JWT_SECRET)  
  req.user = decoded  
  next()  
}
```

Admin-only middleware:

```
const requireAdmin = (req, res, next) => {  
  if (req.user.role !== "ADMIN") return res.status(403).json({ error: "Forbidden" })  
  next()  
}
```

6. Payment Integration (Cashfree)

Cashfree Configuration

Parameter	Value
Secret Key	cfsk_ma_prod_1e7e71e0b9b515312a2190b9bb4ba71c_379c0b52
Environment	PRODUCTION
API Base URL	https://api.cashfree.com/pg
API Version	2023-08-01
Webhook URL	https://api.sparkclean.in/api/payments/webhook
Supported Methods	UPI, GPay, PhonePe, Credit/Debit Cards, Net Banking

Payment Flow

Step 1 – Frontend sends booking data to POST /api/payments/create-order

Step 2 – Backend creates Cashfree order:

POST https://api.cashfree.com/pg/orders

Headers: x-api-version: 2023-08-01

x-client-secret: [CASHFREE_SECRET]

x-client-id: [CASHFREE_APP_ID]

Body: {

order_amount: totalAmount,

order_currency: "INR",

order_id: bookingNumber,

customer_details: { customer_id, customer_name, customer_phone, customer_email }

}

Step 3 – Return payment_session_id to frontend

Step 4 – Frontend opens Cashfree checkout modal

Step 5 – Cashfree sends webhook to /api/payments/webhook on success

Step 6 – Backend verifies signature, updates booking:

```
await prisma.booking.update({
```

```
  where: { id: bookingId },
```

```
  data: { paymentStatus: 'PAID', paymentId: cf_payment_id, status: 'CONFIRMED' }
```

```
})
```

Step 7 – Trigger WhatsApp + Email confirmations

7. Services & Pricing Catalogue

Individual Services

Service Name	Price (INR)	Unit	Category
Bathroom Cleaning	249	per bathroom	INDIVIDUAL
Fridge Cleaning	299	per visit	INDIVIDUAL
Utensils Cleaning	149	per visit	INDIVIDUAL
Kitchen Prep Help	199	per visit	INDIVIDUAL
Dusting & Wiping	299	per visit	INDIVIDUAL
Kitchen Cabinets Cleaning	499	per visit	INDIVIDUAL
Pre-Party Express Clean	799	per visit	INDIVIDUAL
After-Party Cleaning	999	per visit	INDIVIDUAL
Ironing & Folding	1	per cloth	INDIVIDUAL

Combo Packages

Package	Price	Includes	Free Bonus
1 BHK Combo	699	Sweeping & mopping, Dusting & wiping, 1 Bathroom cleaning, Kitchen basic	2 Fans FREE
2 BHK Combo	999	Full house cleaning, 2 Bathrooms, Kitchen cleaning, Dusting & wiping	4 Fans FREE
3 BHK Combo	1499	Full home cleaning, 2-3 Bathrooms, Kitchen deep clean, Balcony basic	6 Fans FREE

8. Booking Flow & Business Logic

Booking Number Generation

```
// Auto-generate unique booking number
// Format: SC-YYYYMMDD-XXXXXX
// Example: SC-20260416-A3F9B2

async function generateBookingNumber(): Promise {
  const date = new Date().toISOString().slice(0,10).replace(/-/g, '')
  const random = Math.random().toString(36).substring(2,8).toUpperCase()
  const num = `SC-${date}-${random}`
  const exists = await prisma.booking.findUnique({ where: { bookingNumber: num } })
  return exists ? generateBookingNumber() : num
}
```

Booking Status State Machine

Status	Trigger	Next States	Action
PENDING	Booking created	CONFIRMED, CANCELLED	Await payment
CONFIRMED	Payment received	ASSIGNED, CANCELLED	Send WhatsApp/email
ASSIGNED	Cleaner assigned by admin	IN_PROGRESS, CANCELLED	Notify customer
IN_PROGRESS	Cleaner starts job	COMPLETED	Update timestamp
COMPLETED	Job finished	— (terminal)	Request review
CANCELLED	Customer or admin cancel	— (terminal)	Process refund if paid

9. Admin Dashboard Backend

GET /api/admin/stats — KPI Response

```
// Prisma aggregation queries for admin dashboard
const [todayBookings, totalRevenue, pendingJobs,
completedJobs, totalCustomers] = await Promise.all([
  prisma.booking.count({
    where: { createdAt: { gte: startOfDayToday() } }
  }),
  prisma.booking.aggregate({
    _sum: { totalAmount: true },
    where: { paymentStatus: 'PAID' }
  }),
  prisma.booking.count({ where: { status: 'PENDING' } }),
  prisma.booking.count({ where: { status: 'COMPLETED' } }),
  prisma.user.count({ where: { role: 'CUSTOMER' } }),
])

// Revenue by day (last 30 days)
const revenueByDay = await prisma.$queryRaw`
SELECT DATE(created_at) as date,
SUM(total_amount) as revenue,
COUNT(*) as bookings
FROM bookings
WHERE payment_status = 'PAID'
AND created_at >= NOW() - INTERVAL '30 days'
GROUP BY DATE(created_at)
ORDER BY date ASC
`
```

10. Notifications (WhatsApp + Email)

WhatsApp Message Template

```
Hi {customerName}! ✦  
Your SparkClean booking is CONFIRMED!  
Booking ID : {bookingNumber}  
Services : {servicesList}  
Date : {scheduledDate}  
Time : {scheduledTime}  
Address : {address}, {area}, Vizag  
Total Paid : Rs.{totalAmount}  
  
Our team will arrive on time. Track your booking at:  
https://sparkclean.in/booking/{bookingId}  
Questions? Call/WhatsApp: 9392420643  
Email: Welcome@vrcpvtltd.com  
Thank you for choosing SparkClean!  
Vizag's Cleanest Choice ✦
```

Email Notification Triggers

Trigger	Recipient	Subject
Booking Created (COD)	Customer	SparkClean Booking Confirmed - {bookingNumber}
Payment Received	Customer + Admin	Payment Confirmed - {bookingNumber}
Cleaner Assigned	Customer	Your cleaner is assigned for {date}
Job Completed	Customer	How was your SparkClean experience?
Booking Cancelled	Customer	Booking {bookingNumber} Cancelled
Daily Report	Admin only	SparkClean Daily Report - {date}
New Contact Message	Admin only	New inquiry from {name}

11. Security & Middleware

Security Layer	Implementation	Purpose
Password Hashing	bcrypt (salt rounds: 12)	Never store plain passwords
JWT Signing	HS256 + JWT_SECRET env var	Tamper-proof auth tokens
Rate Limiting	express-rate-limit (100 req/15min)	Prevent brute force attacks
CORS	Allow only sparkclean.in + localhost	Block unauthorized origins
Helmet.js	HTTP security headers	XSS, clickjacking protection
Input Validation	Zod schemas on all routes	Prevent SQL injection via ORM
Webhook Verification	Cashfree signature check	Prevent fake payment events
SQL Injection	Prisma ORM parameterized queries	Zero raw SQL exposure
Env Variables	.env + Railway secrets	No keys in source code

12. Deployment Architecture

Service	Platform	URL	Cost
Frontend (React)	Vercel	sparkclean.in	Free tier
Backend (Node.js)	Railway.app	api.sparkclean.in	~\$5/mo
PostgreSQL DB	Supabase / Neon	Managed cloud DB	Free tier
File Storage	Cloudinary	res.cloudinary.com	Free tier
Email Service	Resend.com	API based	Free 3k/mo
Domain	GoDaddy / Namecheap	sparkclean.in	~\$12/yr

13. Environment Variables

Backend .env (Railway secrets)

Variable	Value / Description
DATABASE_URL	postgresql://user:pass@host:5432/sparkclean
JWT_SECRET	Random 64-char secret string
JWT_EXPIRES_IN	7d
CASHFREE_APP_ID	From Cashfree dashboard
CASHFREE_SECRET_KEY	cfsk_ma_prod_1e7e71e0b9b515312a2190b9bb4ba71c_379c0b52
CASHFREE_ENV	PRODUCTION
RESEND_API_KEY	From resend.com dashboard
ADMIN_EMAIL	welcome@vrcpvtltd.com
ADMIN_PHONE	9392420643
CLOUDINARY_URL	cloudinary://key:secret@cloudname
FRONTEND_URL	https://sparkclean.in
PORT	3001

Frontend .env (Vercel)

Variable	Value
REACT_APP_API_URL	https://api.sparkclean.in
REACT_APP_CASHFREE_ENV	PRODUCTION
REACT_APP_WA_NUMBER	919392420643

14. Antigravity Build Prompt

Paste the following prompt into Antigravity to build the complete Prisma + PostgreSQL backend autonomously:

```
You are a senior backend engineer. Build a complete production-ready  
REST API backend for SparkClean – Vizag's home cleaning platform.  
Work through all phases autonomously. Do not stop to ask questions.
```

```
# TECH STACK
```

```
- Runtime: Node.js 20 LTS  
- Framework: Express.js with TypeScript  
- ORM: Prisma 5.x  
- Database: PostgreSQL 16  
- Auth: JWT + bcrypt  
- Validation: Zod  
- Email: Resend  
- Payments: Cashfree  
- File Upload: Cloudinary
```

```
# EXISTING FRONTEND
```

```
React 19 + TypeScript + Tailwind CSS (already built)  
Update API calls in: BookingPage, AuthPage, AdminPage, SuccessPage
```

```
# CASHFREE KEYS
```

```
Secret: cfsk_ma_prod_1e7e71e0b9b515312a2190b9bb4ba71c_379c0b52  
Environment: PRODUCTION
```

```
# CONTACT
```

```
Admin email: Welcome@vrcpvtltd.com | Phone: 9392420643
```

```
# BUILD PHASES
```

```
Phase 1 - SCAFFOLD
```

```
mkdir sparkclean-api && cd sparkclean-api  
npm init -y  
npm install express typescript ts-node-dev @types/express  
npm install prisma @prisma/client bcrypt jsonwebtoken zod  
npm install cors helmet express-rate-limit  
npm install resend axios multer cloudinary  
npx tsc --init  
npx prisma init
```

```
Phase 2 - PRISMA SCHEMA
```

```
Write complete schema.prisma with all models:  
User, Booking, Service, Cleaner, PromoCode,  
Review, Notification, ContactMessage  
Run: npx prisma migrate dev --name init  
Run: npx prisma generate
```

```
Phase 3 - SERVER SETUP
```

```
src/index.ts – Express app with all middleware  
src/lib/prisma.ts – PrismaClient singleton  
src/middleware/auth.ts – JWT authenticate + requireAdmin  
src/middleware/validate.ts – Zod request validation
```

Phase 4 - ALL ROUTES

src/routes/auth.ts – register, login, me, forgot-password
src/routes/bookings.ts – CRUD + status updates
src/routes/services.ts – public list + admin CRUD
src/routes/payments.ts – create-order + webhook
src/routes/admin.ts – stats, revenue, customers
src/routes/promo.ts – validate + admin manage

Phase 5 - BUSINESS LOGIC

src/services/bookingService.ts – booking number gen,
status machine, auto-cancel unpaid after 30min
src/services/paymentService.ts – Cashfree order creation,
webhook verification, refund logic
src/services/notificationService.ts – WhatsApp msg format,
email templates via Resend

Phase 6 - SEED DATA

prisma/seed.ts – seed all 12 services + 3 combos,
3 promo codes, 1 admin user
Run: npx prisma db seed

Phase 7 - FRONTEND INTEGRATION

Create src/api/client.ts in React app – Axios instance
with baseURL + JWT interceptor
Update BookingPage to POST /api/bookings
Update AuthPage to POST /api/auth/login and /register
Update AdminPage to GET /api/admin/stats

Phase 8 - DEPLOY

Create railway.json for Railway deployment
Add all environment variables to Railway dashboard
Run: railway up
Confirm API live at api.sparkclean.in

Phase 9 - DOCUMENTATION

Generate API_DOCS.md with all endpoints,
request/response examples, error codes
Build everything now. Do not stop until deployed and tested.

SparkClean PRD v1.0

Vizag's Cleanest Choice — Built by VRC Pvt Ltd

9392420643 | Welcome@vrcpvtltd.com

Confidential — Internal Use Only — April 2026